UNIVERSITY OF LEEDS

COMP1850 Programming

Week 6 Session 2

Image Processing Case Study

Recap

UNIVERSITY OF LEEDS

Last time, we

- Explored the difference between equality of identity and equality of state, and saw how we can test for the latter in our own classes by overriding `__eq__()`
- Saw how classes can extend others through inheritance
- Considered the idea that classes collaborate to solve problems, illustrating this by developing object-oriented software for earthquake data analysis

Today's Aims

UNIVERSITY OF LEEDS

- To revisit an example from previous weeks: reading images from PGM files, processing the images, writing the results to files
- To see how we might apply an object-oriented approach to this scenario, by creating an `Image` class
- To consider some of the design decisions involved in this process
- To cover the broader issue of how we write well-structured and well-documented code

Approach

UNIVERSITY OF LEEDS

- We will spend ~30 mins discussing the issues and explaining some of the code; **pay close attention here!**
- The remainder of the session is open-ended, with several options for things you can develop, building on the code we provide

Design Decision 1

UNIVERSITY OF LEEDS

Support creation of ‘blank slate’ images and creation of images that are loaded from PGM files:

```
class Image:
    ...
    def create_pixels(self, width, height, value=0, maximum=255):
        ...

    def read_pgm(self, filename):
        ...
```

Design Decision 2

UNIVERSITY OF LEEDS

How do we store pixel data?

Linear list? `[0, 0, 0, 0, 0, 0,...]`

List of lists? `[[0, 0, 0,...], [0, 0, 0,...],...]`

(With the ‘list of lists’ approach, the inner lists are individual rows of pixels, the outer list stores those rows in sequence)

Each approach has advantages and disadvantages...

Comparison of Options

UNIVERSITY OF LEEDS

Linear List

- Easier to create
- Pixels can be accessed by a single index – convenient for operations where pixel coordinates are irrelevant
- Code to access a pixel via its 2D coordinates is more complex

List of Lists

- More complex to create
- Always need nested for loops to iterate over all pixels
- Code to access the pixel via 2D coordinates is simple

API vs Implementation

UNIVERSITY OF LEEDS

Key point: the **interface presented to class users can be the same**, irrespective of how we choose to store pixel data

```
# pixel values stored as a list of lists
def get(self, row, col):
    return self.pixels[row][col]

# pixel values stored as a linear list
def get(self, row, col):
    index = row * self.width + col
    return self.pixels[index]
```

create_pixels()

UNIVERSITY OF LEEDS

```
def create_pixels(self, width, height, value=0, maximum=255):
    ...
    if width < 1:
        raise ImageError("width cannot be < 1")
    if height < 1:
        raise ImageError("height cannot be < 1")

    self.width = width
    self.height = height
    self.max_value = maximum
    self.pixels = [value] * (width * height)
```

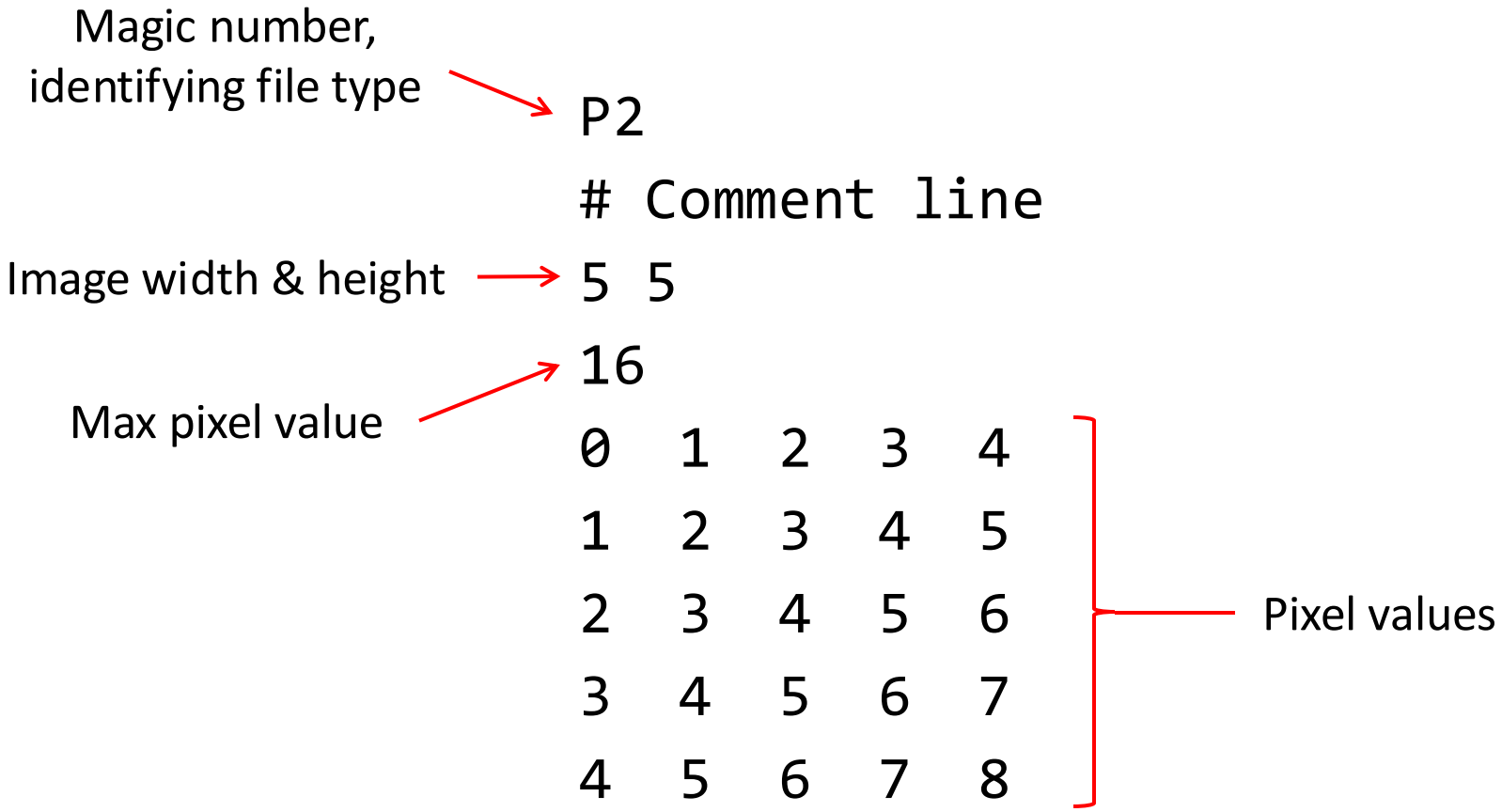
Reminder: PGM Images

UNIVERSITY OF LEEDS

- One of a family of image file formats, not widely used these days, but simple enough that we can work easily with it
- PGM = ‘Portable Grey Map’
- Two versions: one where all of the file is text, which we focus on; another where the header is text but the pixel data are stored in raw binary form
- Related formats are PBM (Portable Bit Map) where pixels are 0 or 1, and PPM, where each pixel has a 3 colour components

PGM Example

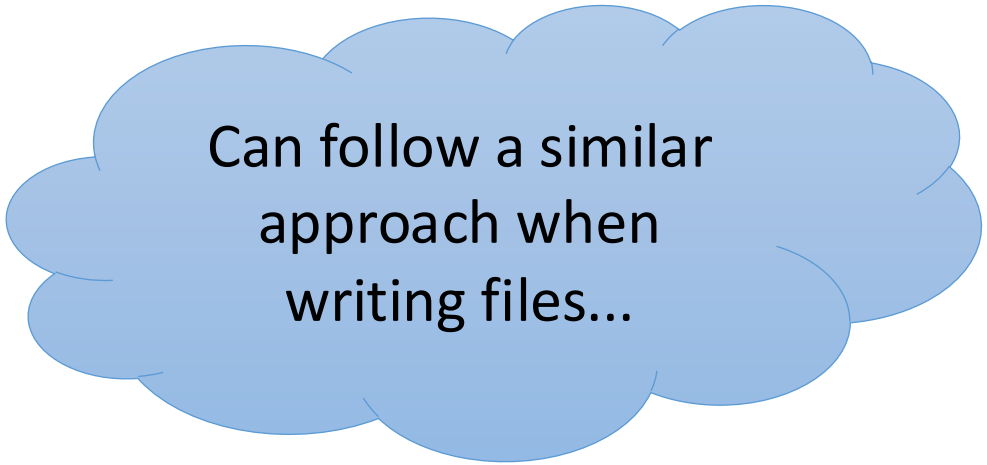
UNIVERSITY OF LEEDS



Design Decision 3

UNIVERSITY OF LEEDS

- Follow a structured approach to reading PGM files
- Subdivide task into reading of header, then reading of pixel data
- Reading of header can be further subdivided into
  - Parse and check magic number (must be P2)
  - Parse image dimensions
  - Parse max value



read_pgm()

UNIVERSITY OF LEEDS

```
def read_pgm(self, filename):
    ...
    with open(filename) as imgfile:
        self._parse_magic_number(imgfile)
        self._parse_dimensions(imgfile)
        self._parse_max_value(imgfile)
        self._parse_pixels(imgfile)
```



Note use of underscore as a prefix

This hides the methods from Python's `help()` function & IDEs

Examine the implementation in `image.py` for more details

Design Decision 4

UNIVERSITY OF LEEDS

- Hard to write a constructor that caters neatly for these two very different ways of creating an image
- Users could simply invoke default constructor and then call either `create_pixels()` or `read_pgm()` on that empty object:

```
img = Image()
img.read_pgm("scene.pgm")
```

- As a convenience, we can wrap these steps up as a **class method**

Class Methods

UNIVERSITY OF LEEDS

```
class Image:
    ...
    @classmethod
    def create(cls, width, height, value=0, maximum=255):
        image = Image()
        image.create_pixels(width, height, value, maximum)
        return image


image = Image.create(100, 100)
```

Note use of class name here!

Over To You...

UNIVERSITY OF LEEDS

- `week1.6/session2/projects` contains various subdirectories, each containing a possible project
- `README.md` in each project subdirectory gives full details of what the project is about
- Examine these, then pick one or two projects to work on for the remainder of this session
  - **Tip**: `lookup_tables`, `filtering` and `pixel_rep` are probably the easiest ones to tackle

End of Session

UNIVERSITY OF LEEDS

Remember:

- `git add` to stage your changes
- `git commit -m` followed by a commit message in quotes, to commit your changes to your Codespace or local repository
- `git push` to send your commits back to github.com